

OmniLitter

Sistema multiplataforma para la gestión y análisis de datos de
contaminación por residuos



Escuela Técnica Superior de
Ingeniería Informática

Trabajo Fin de Grado

Grado: Ingeniería Informática - Ingeniería del Software

Centro: ETSII — Universidad de Sevilla

Curso académico: 2025/26

Autores:

- Alejandro Castilla — alecasbar@alum.us.es
- Ignacio Martínez Díaz — ignmardial@alum.us.es

Tutor académico: Cruz Mata, Fermín

Cotutor: Ortega Rodríguez, Francisco Javier

Cliente / entidad colaboradora: Daniel González Fernández /
Universidad de Cádiz

Índice general

1. Introducción	1
1.1. Contexto y motivación	1
1.2. Problema abordado	1
1.3. Objetivos	2
1.4. Alcance del proyecto	3
2. Contexto y antecedentes	4
2.1. Monitorización de residuos	4
2.2. Aplicaciones de campo y ciencia ciudadana	4
2.3. Herramientas existentes	4
2.4. Taxonomías y datos científicos	4
2.5. Conclusiones del análisis	4
3. Descripción del proyecto	5
3.1. Visión general de OmniLitter	5
3.2. Actores y perfiles de usuario	6
3.3. Plataformas del sistema	8
3.4. Funcionalidades principales	10
3.5. Restricciones y alcance excluido	12
4. Requisitos y análisis	14
4.1. Proceso de toma de requisitos	14
4.2. Requisitos funcionales	15
4.3. Requisitos no funcionales	16
4.4. Requisitos de datos científicos	17
4.5. Casos de uso principales	17
4.6. Trazabilidad entre requisitos y fuentes	17
5. Planificación y gestión	19
5.1. Metodología de trabajo	19
5.2. Herramientas de gestión	19
5.3. Backlog y organización por sprints	20
5.4. Gestión de reuniones	29

5.5. Gestión de riesgos	29
5.6. Gestión del alcance	29
6. Tecnologías utilizadas	30
6.1. Criterios de selección	30
6.2. Portal web	31
6.3. Aplicación móvil	32
6.4. Backend y persistencia	32
6.5. Despliegue y herramientas auxiliares	33
6.6. Alternativas consideradas	33
7. Diseño de la solución	34
7.1. Arquitectura general	34
7.2. Diseño de la base de datos	35
7.3. Diseño de la API REST	39
7.4. Seguridad y control de acceso	40
7.5. Diseño del portal web	41
7.6. Diseño de la aplicación móvil	42
7.7. Sincronización offline	43
8. Implementación	45
8.1. Estructura del repositorio	45
8.2. Implementación del backend	45
8.3. Implementación del portal web	45
8.4. Implementación de la aplicación móvil	45
8.5. Persistencia local y sincronización	45
8.6. Configuración de entornos	45
9. Pruebas y validación	46
9.1. Estrategia de pruebas	46
9.2. Pruebas funcionales	46
9.3. Pruebas de sincronización y offline	46
9.4. Pruebas de permisos y seguridad	46
9.5. Validación con cliente	46
9.6. Matriz requisito-prueba	46
10.Despliegue y operación	47
10.1. Arquitectura de despliegue	47
10.2. Configuración de producción	47
10.3. Despliegue de API y base de datos	47
10.4. Despliegue del portal web	47
10.5. Build y distribución de la aplicación móvil	47
10.6. Limitaciones operativas	47

11. Conclusiones y trabajo futuro	48
11.1. Cumplimiento de objetivos	48
11.2. Limitaciones	48
11.3. Lecciones aprendidas	48
11.4. Líneas de trabajo futuro	48
A. Declaración del uso de IA	50
A.1. Usos realizados	50
A.2. Ejemplos de prompts utilizados	51
A.3. Autoría y revisión crítica	52
B. Requisitos detallados y validación con cliente	53
B.1. Resumen de reuniones con el cliente	53
B.2. Requisitos obtenidos en la reunión inicial	53
B.3. Validación de requisitos mediante mockups	54
B.4. Requisitos funcionales detallados	55
B.5. Requisitos no funcionales detallados	55
B.6. Requisitos de datos científicos	55
C. Manual de instalación y despliegue	57
D. Manual de usuario	58
E. Pruebas completas	59
F. API y diccionario de datos	60
G. Backlog completo	61

Índice de figuras

7.1. Arquitectura general propuesta para OmniLitter.	35
7.2. Diagrama entidad-relación del sistema OmniLitter.	36

Índice de cuadros

4.1. Resumen de requisitos funcionales principales.	16
4.2. Trazabilidad entre reuniones con el cliente y requisitos.	18
5.1. Resumen del backlog inicial integrado en el Sprint 1.	20
5.2. Resumen del backlog del Sprint 2: captura avanzada y flujo offline.	26
6.1. Tecnologías empleadas en los mockups funcionales.	31
6.2. Stack previsto para el portal web.	31
6.3. Stack previsto para la aplicación móvil.	32
6.4. Stack previsto para backend y persistencia.	33
7.1. Descripción de entidades del modelo de datos.	36
7.2. Principales <i>endpoints</i> de la API REST de OmniLitter.	39
7.3. Estados de sincronización y su significado.	44
A.1. Detalle del uso de IA en el trabajo.	51
B.1. Resumen de reuniones mantenidas con el cliente.	53

1. Introducción

1.1. Contexto y motivación

La contaminación por residuos en entornos terrestres y acuáticos constituye uno de los principales problemas del medio ambiente a nivel mundial. La acumulación de residuos en los distintos tipos de entornos genera un gran impacto negativo sobre los ecosistemas, la biodiversidad y la calidad de vida de las personas. Con el objetivo de comprender la magnitud de los hechos y desarrollar estrategias eficaces de prevención y mitigación, ciertas organizaciones, administraciones públicas e instituciones desarrollan programas de monitorización y análisis de residuos en los distintos entornos.

La obtención de datos reales y útiles requiere de la participación de diferentes personas y grupos dispuestos a realizar diversas campañas de monitorización de dichos residuos en los distintos entornos afectados, dejando patentes las observaciones de los residuos para un posterior análisis más exhaustivo. Sin embargo, hoy en día las herramientas para ello se han quedado obsoletas, se basan en capturas y registros manuales muy tediosos o, directamente, no cubren las necesidades actuales.

Teniendo en cuenta lo anterior, el cliente ha planteado la necesidad de desarrollar una herramienta que solucione el problema.

En respuesta a esta necesidad, se propone el desarrollo de OmniLitter. Esta plataforma incluirá tanto un portal web como una aplicación móvil, para la monitorización y registro de residuos para un posterior análisis.

1.2. Problema abordado

Para hacer un seguimiento de los residuos, se necesita que mucha gente colabore, hacer campañas en diferentes sitios y recoger mucha información. Luego, se procesa y analiza toda esta información. Si no se tienen herramientas especializadas, puede haber problemas con la información dispersa, la falta de orden en los registros y las dificultades para unir los datos que obtienen diferentes participantes.

Generalmente, se recoge la información en la naturaleza, donde no siempre hay conexión a Internet. Esto limita el uso de aplicaciones que necesitan estar siempre conectadas a

servidores remotos. Por lo tanto, es necesario tener mecanismos que permitan guardar la información en el dispositivo y sincronizarla después cuando haya conexión a la red.

El proyecto también requiere el uso de clasificaciones de residuos estandarizadas, la asociación de fotos a cada observación y el registro preciso de la ubicación geográfica donde se hace cada muestreo. Hacer todo esto a mano aumenta el riesgo de errores y dificulta el uso posterior de los datos.

El cliente necesita una solución de software que permita gestionar campañas de seguimiento, registrar observaciones geolocalizadas desde dispositivos móviles, operar sin conexión a Internet y centralizar toda la información recopilada en una infraestructura común accesible desde un portal web.

1.3. Objetivos

El objetivo principal de este Trabajo Fin de Grado es diseñar, desarrollar e implementar una plataforma software denominada OmniLitter que facilite la recogida y el posterior análisis de datos relacionados con la monitorización de residuos, de una forma sencilla, eficiente y precisa.

Para alcanzar este objetivo general se establecen los siguientes objetivos específicos:

1. Diseñar una arquitectura software que permita la integración de una aplicación móvil, un portal web y un backend centralizado.
2. Desarrollar una aplicación móvil destinada a la captura de observaciones durante campañas de monitorización.
3. Permitir el funcionamiento de la aplicación móvil en condiciones de conectividad limitada mediante mecanismos de almacenamiento local y sincronización diferida.
4. Registrar información geográfica asociada a cada observación utilizando los sensores de posicionamiento disponibles en los dispositivos móviles.
5. Incorporar la gestión de fotografías y metadatos como evidencias asociadas a las observaciones registradas.
6. Implementar un portal web que facilite la administración de campañas, usuarios y datos recopilados.
7. Centralizar la información obtenida en una base de datos común que garantice la integridad y consistencia de los datos.
8. Utilizar taxonomías estandarizadas para la clasificación de residuos con el fin de favorecer la calidad científica de la información registrada.
9. Aplicar metodologías y herramientas de ingeniería del software para garantizar la calidad, mantenibilidad y escalabilidad de la solución desarrollada.

1.4. Alcance del proyecto

El presente Trabajo Fin de Grado aborda el diseño, desarrollo e implementación de una plataforma software denominada OmniLitter, orientada a facilitar la monitorización y el análisis de la contaminación por residuos en distintos ecosistemas terrestres y acuáticos.

La solución desarrollada está compuesta por una aplicación móvil destinada a la recogida de información en campo, un portal web para la gestión y explotación de los datos recopilados, y un backend centralizado encargado de la lógica de negocio y la persistencia de la información.

Dentro del alcance del proyecto se incluyen las siguientes funcionalidades:

- Diseño e implementación de la arquitectura completa del sistema, incluyendo aplicación móvil, portal web, backend y base de datos.
- Registro de observaciones de residuos utilizando una taxonomía jerárquica estandarizada que garantice la consistencia científica de los datos.
- Captura automática de información geográfica y temporal asociada a cada observación mediante los sensores disponibles en los dispositivos móviles.
- Gestión de fotografías y metadatos asociados a las observaciones registradas.
- Gestión de sesiones de muestreo, incluyendo información contextual como el entorno analizado, la duración de la sesión o las características del área estudiada.
- Funcionamiento en modo offline mediante almacenamiento local de la información y sincronización posterior con el servidor cuando exista conectividad.
- Administración de campañas, usuarios, grupos y permisos mediante un portal web de gestión.
- Visualización de observaciones y campañas mediante mapas interactivos y paneles informativos.
- Exportación de los datos recopilados en formatos adecuados para su posterior explotación y análisis científico.
- Implementación de mecanismos de autenticación, control de acceso y protección de la información almacenada.
- Elaboración de la documentación técnica y académica asociada al desarrollo del sistema.

De esta forma, el alcance del proyecto se centra en proporcionar una plataforma funcional y completa para la recogida, gestión y consulta de datos relacionados con la monitorización de residuos, estableciendo una base tecnológica que permita futuras ampliaciones y desarrollos.

2. Contexto y antecedentes

2.1. Monitorización de residuos

2.2. Aplicaciones de campo y ciencia ciudadana

2.3. Herramientas existentes

2.4. Taxonomías y datos científicos

2.5. Conclusiones del análisis

3. Descripción del proyecto

3.1. Visión general de OmniLitter

OmniLitter es una plataforma multiplataforma desarrollada con el objetivo de facilitar la monitorización y el análisis de residuos en distintos tipos de entornos. El sistema está orientado tanto a investigadores como a colaboradores de campo, proporcionando herramientas que permiten registrar, almacenar y analizar información relacionada con campañas de muestreo de residuos.

La arquitectura de OmniLitter se compone de tres elementos principales estrechamente integrados: una API central (backend), una aplicación móvil y un portal web. La API actúa como núcleo del sistema, gestionando la comunicación entre las distintas plataformas, el acceso a los datos y la aplicación de las reglas de negocio.

La aplicación móvil constituye la herramienta de trabajo utilizada durante las campañas de monitorización. A través de ella, los usuarios pueden participar en grupos de investigación, acceder a campañas de muestreo y registrar sesiones de campo. Durante estas sesiones es posible documentar observaciones de residuos, asociando información como la localización geográfica, la clasificación taxonómica, fotografías y otros datos relevantes para su posterior análisis. Además, la aplicación está diseñada para funcionar tanto en línea como sin conexión, almacenando temporalmente la información en una base de datos local y sincronizándola posteriormente con el servidor cuando exista conexión disponible, automática o manualmente.

Por su parte, el portal web actúa como centro de gestión y análisis de la información recopilada. Desde esta plataforma es posible administrar usuarios, grupos de investigación y campañas de muestreo, así como consultar los datos registrados durante las actividades de campo. Entre sus funcionalidades destaca el módulo de analítica, que permite explorar campañas, sesiones y observaciones mediante diferentes criterios de filtrado, facilitando la obtención de información útil para estudios científicos, proyectos de investigación y procesos de toma de decisiones relacionados con la gestión de residuos.

3.2. Actores y perfiles de usuario

OmniLitter es una plataforma colaborativa en la que distintos usuarios participan en la gestión de campañas de monitorización y en la recogida de observaciones de residuos. Debido a esta naturaleza colaborativa, el sistema implementa un modelo de control de acceso basado en dos conceptos diferenciados: el rol global del usuario dentro de la plataforma y el rol que dicho usuario posee dentro de cada grupo de trabajo.

Esta separación permite que una misma persona pueda desempeñar diferentes responsabilidades según el grupo en el que participe, garantizando al mismo tiempo el aislamiento de la información entre equipos de trabajo independientes.

El acceso a los datos se estructura alrededor de tres elementos fundamentales:

- La cuenta de usuario, que identifica a la persona dentro de la plataforma.
- El grupo de trabajo, que delimita el conjunto de campañas, sesiones y observaciones accesibles.
- El rol asignado, que determina las acciones que el usuario puede realizar sobre dichos datos.

A continuación se describen los principales perfiles contemplados por el sistema.

3.2.1. Visitante

El visitante corresponde a cualquier persona que accede a la parte pública de la plataforma sin haber iniciado sesión. Este perfil únicamente puede consultar información general sobre OmniLitter y acceder a las funcionalidades de autenticación, como el inicio de sesión, el registro de una nueva cuenta o la recuperación de contraseña.

No dispone de acceso a campañas, observaciones, métricas ni información perteneciente a los grupos de trabajo.

3.2.2. Usuario autenticado

El usuario autenticado constituye el perfil base del sistema. Se trata de cualquier persona que dispone de una cuenta válida y ha iniciado sesión correctamente.

Este perfil puede gestionar la información asociada a su propia cuenta, consultar los grupos a los que pertenece, seleccionar un grupo activo y aceptar o rechazar invitaciones recibidas. Sin embargo, los permisos operativos sobre campañas y observaciones dependen del rol que posea dentro de cada grupo.

3.2.3. Investigador de campo

El investigador de campo es el perfil responsable de la recogida de información durante las campañas de monitorización. Su actividad principal se desarrolla mediante la aplicación móvil, desde la cual puede crear sesiones de muestreo, registrar observaciones, clasificar residuos mediante la taxonomía disponible, adjuntar fotografías y capturar la localización geográfica de cada registro.

Además, este perfil puede trabajar en modo offline cuando no existe conectividad, almacenando localmente la información para sincronizarla posteriormente con el servidor.

Aunque puede consultar la información necesaria para desarrollar su trabajo, no dispone de permisos para administrar usuarios, gestionar campañas o modificar la estructura organizativa del grupo.

3.2.4. Administrador de grupo

El administrador de grupo es el responsable de coordinar la actividad de un grupo de trabajo concreto. Este perfil puede gestionar los miembros del grupo, enviar invitaciones, asignar roles de membresía y administrar las campañas asociadas a dicho grupo.

Entre sus funciones se encuentran la creación, edición y organización de campañas de monitorización, así como la supervisión de la actividad realizada por los investigadores de campo. También dispone de acceso a métricas agregadas e información de seguimiento relacionada con las campañas del grupo.

Sus permisos se limitan exclusivamente a los grupos en los que posee dicho rol. Por tanto, un mismo usuario puede actuar como administrador en un grupo y como investigador de campo en otro diferente.

3.2.5. Superadministrador

El superadministrador representa el perfil con mayor nivel de autoridad dentro de OmniLitter. Su función principal consiste en administrar la plataforma a nivel global y garantizar su correcto funcionamiento.

Este perfil puede gestionar usuarios, modificar roles globales, supervisar grupos de trabajo y realizar tareas de mantenimiento o soporte. A diferencia del administrador de grupo, sus permisos no se encuentran restringidos a un único grupo, sino que abarcan la totalidad del sistema.

Debido a su alcance, este perfil está destinado a tareas de administración y soporte de la plataforma, y no a la realización de actividades habituales de monitorización en campo.

3.2.6. Usuario sin grupo activo

Finalmente, OmniLitter contempla la situación de usuarios registrados que todavía no pertenecen a ningún grupo o que no han seleccionado uno como grupo activo.

En este estado, el usuario puede gestionar su cuenta personal y revisar invitaciones pendientes, pero no tiene acceso a campañas, observaciones ni funcionalidades relacionadas con la monitorización hasta incorporarse a un grupo de trabajo.

3.2.7. Actores de apoyo

Además de los usuarios que interactúan directamente con la plataforma, OmniLitter depende de diversos componentes técnicos que permiten el funcionamiento del sistema.

Entre ellos destacan el servicio de autenticación encargado de validar credenciales y gestionar sesiones, el servicio de correo utilizado para determinadas comunicaciones automáticas, los dispositivos móviles empleados durante las campañas de campo y la infraestructura backend responsable de la lógica de negocio y el almacenamiento persistente de la información.

Aunque estos elementos no representan usuarios propiamente dichos, constituyen actores fundamentales para la correcta ejecución de los procesos definidos por la plataforma.

3.3. Plataformas del sistema

OmniLitter se concibe como una plataforma compuesta por varios elementos que colaboran entre sí para cubrir las distintas fases del proceso de monitorización de residuos. La naturaleza del problema exige disponer de herramientas adaptadas tanto al trabajo de campo como a la gestión y análisis posterior de la información recopilada.

Para dar respuesta a estas necesidades, el sistema se estructura en tres componentes principales: una aplicación móvil, un portal web y un backend centralizado. Esta separación permite adaptar cada componente a las tareas específicas que debe desempeñar, manteniendo al mismo tiempo una gestión unificada de la información.

3.3.1. Aplicación móvil

La aplicación móvil constituye la herramienta principal para la recogida de datos en campo. Está orientada a los usuarios que participan directamente en las campañas de monitorización y les permite registrar observaciones de residuos en el lugar donde se producen.

A través de esta aplicación es posible consultar campañas, iniciar sesiones de muestreo, registrar observaciones, capturar fotografías asociadas a los residuos encontrados y almacenar la localización geográfica de cada registro. Además, la aplicación está diseñada para funcionar en entornos con conectividad limitada, permitiendo continuar el trabajo incluso cuando no existe acceso a Internet y sincronizando posteriormente la información con el servidor.

3.3.2. Portal web

El portal web proporciona las herramientas necesarias para la gestión y supervisión de la información recopilada. Esta plataforma está orientada principalmente a administradores de grupo, investigadores y superadministradores.

Desde el portal web es posible gestionar usuarios y grupos de trabajo, crear y administrar campañas de monitorización, consultar observaciones registradas, visualizar información geográfica mediante mapas interactivos y acceder a métricas agregadas sobre los datos recopilados. Asimismo, permite exportar la información para su utilización en procesos posteriores de análisis científico.

A diferencia de la aplicación móvil, cuyo objetivo principal es facilitar la recogida de datos en campo, el portal web está enfocado a la coordinación, revisión y explotación de la información almacenada.

3.3.3. Backend y almacenamiento centralizado

El backend actúa como elemento de coordinación entre las distintas plataformas del sistema. Su función consiste en centralizar la lógica de negocio, gestionar la autenticación de usuarios, controlar los permisos de acceso y garantizar la consistencia de los datos almacenados.

Tanto la aplicación móvil como el portal web se comunican con este componente para consultar o registrar información. De esta forma, todas las operaciones se realizan sobre una única fuente de datos común, evitando inconsistencias y facilitando la gestión centralizada del sistema.

La información recopilada durante las campañas se almacena en una base de datos centralizada que actúa como repositorio principal de usuarios, grupos, campañas, sesiones de muestreo, observaciones y fotografías asociadas.

3.3.4. Integración de las plataformas

La combinación de estas plataformas permite cubrir el ciclo completo de trabajo de una campaña de monitorización. La aplicación móvil facilita la recogida de datos sobre el

terreno, el backend garantiza la gestión y consistencia de la información, y el portal web proporciona las herramientas necesarias para su consulta, supervisión y análisis posterior.

Esta arquitectura permite que cada componente se especialice en una función concreta, favoreciendo la mantenibilidad del sistema y facilitando su evolución futura.

3.4. Funcionalidades principales

OmniLitter proporciona un conjunto de funcionalidades orientadas a cubrir el ciclo completo de una campaña de monitorización de residuos, desde la organización de los equipos de trabajo hasta la recogida y posterior consulta de los datos obtenidos.

3.4.1. Gestión de usuarios y grupos

La plataforma permite el registro y autenticación de usuarios, así como la organización de estos en grupos de trabajo independientes. Cada grupo dispone de sus propias campañas, sesiones y observaciones, garantizando el aislamiento de la información entre equipos.

Además, el sistema incorpora distintos niveles de permisos que permiten diferenciar entre usuarios de campo, administradores de grupo y administradores globales de la plataforma.

3.4.2. Gestión de campañas

OmniLitter permite crear y administrar campañas de monitorización asociadas a un grupo de trabajo. Cada campaña define el contexto en el que se desarrollarán las actividades de muestreo, incluyendo información descriptiva, fechas de realización y características generales del entorno estudiado.

Las campañas actúan como elemento organizador de toda la información recogida durante el trabajo de campo.

3.4.3. Sesiones de muestreo

Dentro de cada campaña, los usuarios pueden crear sesiones de muestreo que representan actividades concretas de recogida de datos.

Las sesiones permiten registrar información contextual relacionada con el trabajo realizado, como el entorno de estudio, el método de muestreo empleado, la duración de la actividad o la ubicación geográfica asociada.

Esta funcionalidad aporta trazabilidad a los datos recopilados, permitiendo conocer las condiciones bajo las que se registró cada observación.

3.4.4. Registro de observaciones

El registro de observaciones constituye la funcionalidad principal del sistema. Durante una sesión de muestreo, los usuarios pueden documentar los residuos encontrados indicando su localización, fecha de registro y clasificación correspondiente.

Cada observación queda vinculada a una campaña y una sesión concretas, permitiendo mantener la coherencia y organización de los datos almacenados.

3.4.5. Clasificación mediante taxonomías

La plataforma incorpora una taxonomía de residuos que permite clasificar las observaciones de forma homogénea y estandarizada.

El uso de categorías comunes mejora la calidad de los datos recopilados y facilita su comparación entre campañas, grupos de trabajo y periodos temporales diferentes.

3.4.6. Registro fotográfico y geolocalización

Las observaciones pueden complementarse mediante fotografías y coordenadas geográficas obtenidas desde el dispositivo móvil.

Estas evidencias permiten contextualizar cada registro, aumentar la fiabilidad de los datos y facilitar posteriores procesos de revisión y análisis.

3.4.7. Trabajo offline y sincronización

La aplicación móvil ha sido diseñada para utilizarse en entornos donde la conectividad puede ser limitada o inexistente.

Por este motivo, los usuarios pueden continuar registrando observaciones sin conexión a Internet. Cuando la conectividad se restablece, la información almacenada localmente se sincroniza con el sistema central, garantizando la disponibilidad de los datos para el resto de usuarios autorizados.

3.4.8. Consulta y visualización de información

El portal web permite consultar la información recopilada durante las campañas mediante diferentes mecanismos de visualización.

Los usuarios autorizados pueden revisar observaciones, consultar información geográfica a través de mapas interactivos y acceder a métricas generales relacionadas con la actividad de monitorización realizada.

3.4.9. Exportación de datos

OmniLitter incorpora mecanismos para exportar la información recopilada en formatos adecuados para su tratamiento posterior.

Esta funcionalidad facilita la reutilización de los datos en herramientas externas de análisis y favorece su utilización en estudios científicos relacionados con la contaminación por residuos.

3.4.10. Resumen funcional

De forma resumida, OmniLitter permite:

- Gestionar usuarios, grupos y permisos.
- Planificar campañas de monitorización.
- Realizar sesiones de muestreo en campo.
- Registrar observaciones geolocalizadas.
- Clasificar residuos mediante una taxonomía estandarizada.
- Asociar fotografías y metadatos a las observaciones.
- Trabajar sin conexión y sincronizar posteriormente los datos.
- Consultar la información recopilada mediante mapas y métricas.
- Exportar los datos para su análisis posterior.

3.5. Restricciones y alcance excluido

Con el objetivo de acotar el alcance del proyecto y ajustarlo a las limitaciones temporales y de recursos propias de un Trabajo Fin de Grado, se decidió excluir una serie de funcionalidades y actividades que, aunque podrían aportar valor a la plataforma, no resultaban imprescindibles para cumplir los objetivos principales establecidos.

Las principales exclusiones son las siguientes:

- **Soporte multiidioma avanzado.** Aunque la arquitectura de la aplicación permite la internacionalización de la interfaz, el alcance del proyecto se limita al español y al inglés. La incorporación de idiomas adicionales requeriría un esfuerzo significativo de traducción y mantenimiento, además de depender en algunos casos de servicios externos incompatibles con los requisitos de funcionamiento offline de la aplicación móvil.

- **Mantenimiento evolutivo y correctivo posterior a la entrega.** El presente trabajo contempla el análisis, diseño, implementación y validación de la plataforma. Las actividades relacionadas con el mantenimiento posterior, incorporación de nuevas funcionalidades o soporte continuado tras la finalización del proyecto quedan fuera de su alcance.
- **Identificación automática de residuos mediante inteligencia artificial.** Durante las fases iniciales del proyecto se consideró la posibilidad de incorporar mecanismos de reconocimiento automático de residuos a partir de imágenes. Sin embargo, el desarrollo, entrenamiento y validación de modelos de visión artificial requiere una inversión de tiempo, recursos computacionales y conjuntos de datos especializados que exceden las posibilidades de este Trabajo Fin de Grado.
- **Publicación oficial en tiendas de aplicaciones.** Aunque la aplicación móvil ha sido desarrollada para poder ejecutarse en dispositivos reales, la publicación oficial en plataformas como Google Play Store o Apple App Store no forma parte del alcance del proyecto. Este proceso implica requisitos administrativos, costes económicos y procedimientos de revisión que no resultan necesarios para la validación académica de la solución desarrollada.

Estas exclusiones permiten concentrar los esfuerzos de desarrollo en las funcionalidades esenciales de OmniLitter, garantizando una solución completa y funcional dentro de las restricciones temporales y organizativas del proyecto.

4. Requisitos y análisis

Este capítulo recoge el proceso seguido para obtener, contrastar y refinar los requisitos de OmniLitter. Dado que el proyecto parte de una necesidad real planteada por un cliente experto en el dominio, la definición de requisitos no se limitó a una fase inicial cerrada, sino que evolucionó a partir de reuniones de toma de requisitos, validación de mockups y revisión de los flujos propuestos.

4.1. Proceso de toma de requisitos

La toma de requisitos se realizó en dos fases principales:

1. **Reunión inicial de toma de requisitos.** En esta primera reunión se presentó el problema: renovar y ampliar una herramienta existente de monitorización de residuos, incorporando nuevos entornos de muestreo, gestión de usuarios y grupos, funcionamiento sin conexión y un portal web de consulta.
2. **Reunión de contraste con mockups.** Tras construir los primeros prototipos interactivos del portal web y de la aplicación móvil, se revisó con el cliente si los flujos propuestos cubrían los requisitos inicialmente identificados. Esta reunión sirvió para validar el enfoque general y detectar qué partes debían priorizarse o explicarse con mayor detalle en el diseño.

Esta forma de trabajo permitió reducir la incertidumbre del proyecto: primero se recogieron las necesidades de alto nivel y posteriormente se contrastaron con pantallas navegables antes de abordar la implementación definitiva.

4.1.0.1. Reunión inicial con el cliente

En la reunión inicial se identificaron las necesidades fundamentales del sistema. El cliente planteó que la nueva solución debía superar el alcance de la aplicación existente, centrada principalmente en residuos flotantes, para permitir la captura de datos en distintos entornos naturales.

Los puntos principales extraídos de esta reunión fueron:

- El sistema debe estar formado por una **aplicación móvil** para trabajo de campo y un **portal web** para consulta, gestión y análisis.
- La aplicación móvil debe permitir registrar observaciones de residuos en entornos acuáticos y terrestres, incluyendo ríos, mares, playas y bosques.
- Cada observación debe guardar, siempre que sea posible, posición GPS, fecha, hora, tipo de residuo, cantidad y fotografía.
- La aplicación debe funcionar sin conexión, ya que el trabajo de campo puede realizarse en zonas con cobertura limitada.
- El sistema debe permitir trabajar con grupos de investigación y roles de usuario, evitando que los datos de un grupo sean visibles por otros grupos.
- El portal web debe permitir visualizar los datos mediante listados, mapas y métricas agregadas.
- La taxonomía de residuos debe estar organizada de forma navegable y ser suficientemente flexible para ampliarse en el futuro.

4.1.0.2. Reunión de validación de mockups

Una vez generados los mockups funcionales, se realizó una segunda reunión para contrastar con el cliente si los flujos planteados respondían correctamente a sus necesidades. En esta revisión se comprobó que los prototipos permitían representar los procesos principales del sistema: autenticación, selección de grupo, creación de sesiones de muestreo, registro de observaciones, consulta de mapas y visualización de datos desde el portal.

La reunión no introdujo una lista cerrada de nuevos requisitos funcionales, pero sí sirvió para confirmar la importancia de tres aspectos que debían mantenerse en el diseño final:

- La aplicación móvil debía seguir siendo prioritaria para el trabajo de campo y conservar el enfoque *offline-first*.
- El portal web debía mantener una separación clara entre datos de distintos grupos de investigación.
- La captura de datos debía preservar metadatos científicos suficientes para que las observaciones pudieran analizarse posteriormente.

4.2. Requisitos funcionales

Los requisitos funcionales definen el comportamiento esperado del sistema. A continuación se resumen los más relevantes para el alcance del proyecto:

Cuadro 4.1: Resumen de requisitos funcionales principales.

ID	Descripción	Prioridad
RF-001	Inicio de sesión en portal web y aplicación móvil.	Crítica
RF-002	Cierre de sesión y revocación de sesión local.	Alta
RF-004	Soporte de roles y control de acceso por grupo.	Crítica
RF-005	Creación de grupos y asociación de usuarios a grupos.	Crítica
RF-006	Selección de grupo activo para filtrar datos y acciones.	Alta
RF-007	Trabajo por sesiones de muestreo en la aplicación móvil.	Crítica
RF-008	Captura de coordenadas GPS al iniciar sesiones de muestreo y observaciones.	Crítica
RF-010	Adjuntar fotografías a una observación.	Alta
RF-012	Navegación por taxonomía jerárquica de residuos.	Crítica
RF-015	Registro de cantidad observada como entero positivo.	Crítica
RF-020	Revisión de sesión antes de finalizarla.	Alta
RF-022	Funcionamiento sin conexión durante la captura de campo.	Crítica
RF-023	Cola de sincronización y subida automática al recuperar conectividad.	Crítica
RF-027	Listado de sesiones y observaciones con filtros en el portal.	Alta
RF-028	Visualización de observaciones en mapa.	Alta
RF-030	Dashboard con métricas agregadas.	Media
RF-032	Gestión de usuarios desde el portal.	Alta
RF-033	Gestión de grupos y membresías desde el portal.	Alta
RF-035	Soporte de interfaz en español e inglés.	Media

4.3. Requisitos no funcionales

Los requisitos no funcionales recogen restricciones de calidad, seguridad y operación. Los más relevantes son:

- **Seguridad:** toda la comunicación entre clientes y backend debe realizarse mediante HTTPS/TLS.
- **Aislamiento de datos:** los usuarios solo podrán acceder a datos de los grupos a los que pertenecen.
- **Offline-first:** las operaciones de captura en campo no deben depender de conectividad.
- **Fiabilidad de sincronización:** los datos no deben perderse ante cortes de red o cierres inesperados de la aplicación.
- **Usabilidad:** la aplicación móvil debe estar optimizada para uso en campo, con botones claros, formularios rápidos y navegación simple.
- **Trazabilidad:** el sistema debe mantener logs de operaciones relevantes y relacionar requisitos con pruebas.

4.4. Requisitos de datos científicos

Al tratarse de una herramienta para captura y análisis de datos científicos, el sistema debe preservar suficiente información contextual para que los datos sean interpretables y reproducibles. En particular:

- La taxonomía debe presentarse de forma jerárquica sin perder el código identificador de cada tipo de residuo.
- Las sesiones deben almacenar metadatos de muestreo como entorno, ancho de transecto, duración, protocolo, versión de aplicación y versión de taxonomía.
- Las fotografías deben conservar metadatos relevantes cuando estén disponibles, como fecha de captura, dimensiones o coordenadas EXIF.
- Los datos exportados deben incluir campos suficientes para análisis posteriores y, cuando sea posible, un diccionario de columnas y unidades.
- La versión de taxonomía utilizada debe quedar asociada a las sesiones y observaciones para garantizar reproducibilidad.

4.5. Casos de uso principales

4.6. Trazabilidad entre requisitos y fuentes

La tabla 4.2 resume la relación entre las reuniones con el cliente y los requisitos derivados.

Cuadro 4.2: Trazabilidad entre reuniones con el cliente y requisitos.

Fuente		Necesidad identificada	Requisitos relacionados
Reunión inicial		Aplicación móvil de campo y portal web de gestión.	RF-001, RF-027, RF-030
Reunión inicial		Captura de observaciones con GPS, fecha, tipo de residuo, cantidad y fotografía.	RF-008, RF-010, RF-012, RF-015
Reunión inicial		Funcionamiento sin conexión y sincronización posterior.	RF-022, RF-023, fiabilidad de sincronización
Reunión inicial		Gestión de grupos, usuarios y privacidad de datos.	RF-004, RF-005, RF-006, RF-032, RF-033, aislamiento de datos
Validación mockups	de	Confirmación del enfoque <i>offline-first</i> y de la captura por sesiones.	RF-007, RF-020, RF-022, RF-023
Validación mockups	de	Confirmación de la separación por grupos y la trazabilidad científica.	RF-004, RF-006, requisitos de datos científicos

El detalle ampliado de requisitos y de su validación con mockups se incluye en el Anexo B.

5. Planificación y gestión

El desarrollo de OmniLitter se ha planteado de forma iterativa, combinando tareas de análisis, diseño, prototipado e implementación. Esta aproximación resulta adecuada para un proyecto con cliente real, ya que permite validar decisiones antes de invertir esfuerzo en una implementación definitiva.

5.1. Metodología de trabajo

La metodología seguida se basa en ciclos cortos de trabajo:

1. Recogida de necesidades con el cliente y tutores.
2. Definición de requisitos funcionales, no funcionales y de datos científicos.
3. Elaboración de mockups funcionales para validar flujos de usuario.
4. Revisión con el cliente y refinamiento del prototipo.
5. Diseño de la arquitectura, base de datos y componentes del sistema.
6. Implementación progresiva de las partes finales del proyecto.

Aunque no se ha aplicado una metodología ágil formal completa, el proyecto adopta una forma de trabajo incremental: cada avance genera un artefacto revisable (documento de requisitos, mockup, diagrama, módulo de código o capítulo de memoria) que se contrasta antes de continuar.

5.2. Herramientas de gestión

Para trasladar esta aproximación a la ejecución diaria del proyecto se ha utilizado un backlog funcional agrupado por sprints. Cada elemento del backlog representa una capacidad observable del sistema y se ha ordenado según sus dependencias técnicas y funcionales. De este modo, las tareas iniciales no se han tratado como actividades aisladas, sino como una secuencia de construcción progresiva: primero se establece la infraestructura común de autenticación y permisos, después se consolida la navegación de los clientes y, finalmente, se habilitan los primeros flujos de captura y persistencia de datos.

5.3. Backlog y organización por sprints

La planificación del desarrollo se ha estructurado mediante un backlog funcional priorizado. Este backlog recoge las funcionalidades necesarias para transformar los requisitos validados en la Sección 4 en incrementos implementables, manteniendo la coherencia con la arquitectura de tres capas definida para OmniLitter: portal web, aplicación móvil y backend REST con persistencia relacional.

5.3.0.1. Sprint 1: Infraestructura, autenticación y flujo básico de captura

El primer bloque de trabajo corresponde al Sprint 1 y concentra las tareas T01 a T10. Estas tareas constituyen la base inicial del sistema, ya que introducen la configuración funcional común, la autenticación, el control de acceso por grupos, la navegación del portal web, la administración inicial, el modelo de sesiones de muestreo, la taxonomía y el flujo básico de creación de observaciones desde la aplicación móvil. Su selección responde a una decisión de planificación: antes de incorporar módulos analíticos, exportaciones avanzadas o sincronización completa, era necesario disponer de una estructura mínima que conectara usuarios, grupos, sesiones, observaciones y clientes.

Cuadro 5.1: Resumen del backlog inicial integrado en el Sprint 1.

ID	Funcionalidad	Área	Dependencias
T01	Login y logout común en app y portal	Backend, web y app	Ninguna
T02	Roles, permisos y grupo activo	Backend y web	T01
T03	Portal web: layout, navegación y rutas protegidas	Web	T01, T02
T04	Portal admin: usuarios, grupos y membresías	Web y backend	T02, T03
T05	Campañas y sesiones de muestreo en backend	Backend	T02
T06	App móvil: navegación principal y perfil	App móvil	T01
T07	App móvil: taxonomía jerárquica, búsqueda y favoritos	App móvil y backend	T06, T01
T08	App móvil: creación de sesión de muestreo	App móvil y backend	T05, T06
T09	API de observaciones y adjuntos para captura de campo	Backend	T05, T07
T10	App móvil: formulario de observación	App móvil	T07, T08, T09

La tabla anterior se incluye únicamente como síntesis de orientación. La planificación real del proyecto se entiende a partir de la relación entre las tareas, sus dependencias y el impacto que tienen en el producto final, como se describe en los apartados siguientes.

T01 – Login y logout común en app y portal

La primera tarea tiene como objetivo funcional establecer un mecanismo común de inicio y cierre de sesión para los dos clientes del sistema: el portal web y la aplicación móvil. Su papel dentro del backlog es fundacional, ya que permite identificar al usuario antes de aplicar permisos, seleccionar grupos de trabajo o registrar datos científicos asociados a una persona concreta.

En el estado actual documentado del proyecto, esta tarea se materializa en los flujos de autenticación representados en los mockups y en el diseño de la API REST, donde se prevén los puntos de entrada de autenticación, renovación de sesión y cierre de sesión mediante tokens JWT. En el portal web se contempla la existencia de rutas públicas y protegidas, mientras que en la aplicación móvil el acceso al perfil y a las pantallas principales parte igualmente de una sesión activa. La dependencia de esta tarea es nula, por lo que actúa como punto de arranque del resto del Sprint 1.

Su impacto en el producto final es directo: proporciona la base de seguridad necesaria para que OmniLitter pueda diferenciar usuarios, asociar observaciones a investigadores concretos y mantener una experiencia coherente entre web y móvil. Además, permite que las tareas posteriores no dupliquen lógica de identificación en cada capa.

T02 – Roles, permisos y grupo activo

La segunda tarea persigue introducir el modelo de autorización basado en roles y grupos. Su objetivo funcional es que el sistema no solo conozca qué usuario ha iniciado sesión, sino también qué permisos tiene dentro de cada grupo de investigación y cuál es el grupo activo sobre el que se filtran las operaciones.

La descripción incluida en la memoria se corresponde con el modelo de membresías definido en el diseño de datos, donde el rol no pertenece directamente al usuario, sino a su relación con un grupo determinado. Esta decisión permite que una misma persona pueda actuar como administradora en un grupo y como colaboradora en otro. En el portal web, el concepto de grupo activo condiciona la información mostrada en el dashboard y en los listados, mientras que las secciones de administración se reservan a perfiles con permisos suficientes.

La tarea depende de T01, porque la asignación de permisos solo puede aplicarse a usuarios autenticados. Su impacto es especialmente relevante para el producto final: garantiza el aislamiento de datos entre equipos de investigación, permite cumplir los requisitos de privacidad por grupo y prepara la base para la administración de usuarios, campañas y sesiones.

T03 – Portal web: layout, navegación y rutas protegidas

Esta tarea tiene como objetivo funcional construir la estructura navegable del portal web. En términos de planificación, representa el paso desde la autenticación y el control de acceso hacia una interfaz organizada, capaz de separar las páginas públicas de las secciones internas del sistema.

El diseño documentado contempla un portal web de tipo *Single Page Application* con React, TypeScript, Vite y React Router. La navegación se organiza mediante rutas públicas, como la página de inicio y el formulario de acceso, y rutas protegidas bajo el área interna de la aplicación. Estas rutas protegidas verifican la existencia de una sesión activa y redirigen al usuario si no dispone de acceso. El layout general integra elementos persistentes, como barra lateral, cabecera y selector de grupo activo, que sirven de soporte para las pantallas posteriores.

T03 depende de T01 y T02, ya que la navegación protegida necesita autenticación y debe adaptarse a los permisos del usuario. Su impacto en el producto final consiste en proporcionar una experiencia web consistente, extensible y preparada para alojar el dashboard, los listados, la administración y los módulos de análisis que se incorporen en etapas posteriores.

T04 – Portal admin: usuarios, grupos y membresías

La cuarta tarea amplía el portal web con capacidades administrativas. Su objetivo funcional es permitir la gestión de usuarios, grupos de investigación y membresías, de forma que la estructura organizativa del sistema pueda mantenerse desde el propio portal sin intervención directa sobre la base de datos.

El diseño documentado se apoya en el modelo de datos formado por usuarios, grupos y membresías, así como en el diseño de endpoints de la API para listar, crear, actualizar o archivar grupos y gestionar miembros. En la interfaz web, estas funciones se vinculan al rol de administrador y se ocultan a usuarios colaboradores, manteniendo la separación de responsabilidades dentro del sistema.

Esta tarea depende de T02, porque la administración solo tiene sentido una vez definido el modelo de roles, y de T03, porque necesita integrarse dentro de la navegación protegida del portal. Su impacto final es permitir que OmniLitter sea operable por equipos reales, facilitando la incorporación de investigadores, la creación de grupos y la asignación de permisos sin romper el aislamiento de datos.

T05 – Campañas y sesiones de muestreo en backend

La quinta tarea introduce en el backend las entidades que estructuran la actividad científica del sistema: campañas y sesiones de muestreo. Su objetivo funcional es representar el

contexto en el que se capturan las observaciones, diferenciando el proyecto o campaña de investigación de cada salida concreta de campo.

En la documentación actual, esta tarea se refleja en el modelo relacional mediante las entidades `CAMPAIGN` y `SURVEY_SESSION`, y en el contrato de API previsto para crear, consultar y actualizar sesiones. Las sesiones almacenan metadatos relevantes como entorno, parámetros de muestreo, posición inicial, duración, versión de taxonomía y estado de sincronización. Esta información resulta necesaria para que los datos registrados sean interpretables y reproducibles.

T05 depende de T02, ya que las campañas y sesiones deben quedar asociadas a grupos y usuarios con permisos adecuados. Su impacto en el producto final es central: convierte la captura de residuos en un proceso científicamente contextualizado y prepara las dependencias necesarias para la creación de observaciones y adjuntos.

T06 – App móvil: navegación principal y perfil

La sexta tarea traslada la base de autenticación a la aplicación móvil. Su objetivo funcional es establecer la navegación principal de la app y una pantalla de perfil que represente al usuario autenticado, sirviendo como punto de entrada para las funcionalidades de campo.

El diseño documentado plantea una aplicación móvil con React Native y Expo, organizada mediante navegación de primer nivel y flujos de pila. Las secciones principales se estructuran alrededor de la actividad de campo: inicio, sesiones, mapa, sincronización y perfil. Esta organización permite separar las acciones de captura de datos de las pantallas informativas y de configuración, manteniendo una interfaz adecuada para su uso en condiciones reales de muestreo.

T06 depende de T01 porque la navegación móvil parte de una sesión válida. Su impacto en el producto final consiste en proporcionar una estructura estable para que el usuario pueda iniciar sesiones, consultar su estado y acceder posteriormente a la taxonomía, la captura de observaciones y la sincronización.

T07 – App móvil: taxonomía jerárquica, búsqueda y favoritos

La séptima tarea tiene como objetivo funcional incorporar en la aplicación móvil la taxonomía de residuos de forma navegable, con soporte para búsqueda y elementos favoritos. Esta funcionalidad resulta crítica porque la selección del tipo de residuo es uno de los datos principales de cada observación.

La memoria describe una taxonomía jerárquica basada en códigos estables y preparada para funcionar sin conexión. En el diseño de la aplicación móvil, dicha taxonomía se presenta mediante una navegación por categorías y elementos, complementada por mecanismos que

agilizan la captura en campo, como favoritos. En el backend, la taxonomía queda modelada mediante la entidad `TAXONOMY_ITEM` y endpoints de consulta, de modo que pueda descargarse y asociarse a sesiones y observaciones con su versión correspondiente.

T07 depende de T06, porque se integra en la navegación móvil, y de T01, porque la descarga o disponibilidad de la taxonomía se vincula al contexto del usuario. Su impacto en el producto final es doble: mejora la usabilidad durante el trabajo de campo y refuerza la calidad científica de los datos al mantener una clasificación controlada y trazable.

T08 – App móvil: creación de sesión de muestreo

La octava tarea permite iniciar desde la aplicación móvil una nueva sesión de muestreo. Su objetivo funcional es que el investigador pueda registrar el contexto de la salida de campo antes de introducir observaciones individuales.

El diseño documentado define un flujo de creación de sesión en el que se recogen datos como nombre, tipo de entorno, parámetros del transecto y coordenadas de inicio cuando estén disponibles. Esta información se relaciona con el diseño de persistencia local mediante SQLite, pensado para que las sesiones puedan crearse sin conexión y sincronizarse posteriormente con el backend. En la arquitectura general, la sesión actúa como contenedor de las observaciones generadas durante una jornada o tramo de muestreo.

T08 depende de T05, porque necesita el modelo backend de campañas y sesiones, y de T06, porque se integra dentro de la navegación móvil. Su impacto en el producto final es esencial: habilita el trabajo de campo de forma ordenada y permite asociar cada observación a un contexto metodológico claro.

T09 – API de observaciones y adjuntos para captura de campo

La novena tarea se centra en la capa backend y tiene como objetivo funcional proporcionar la API necesaria para registrar observaciones y asociarles adjuntos, principalmente fotografías tomadas durante la captura de campo.

En el diseño actual, esta tarea se refleja en las entidades `OBSERVATION` y `PHOTO`, así como en los endpoints previstos para crear observaciones, listar observaciones de una sesión y subir fotografías mediante peticiones multipart. Las observaciones incluyen el tipo de residuo, cantidad, atributos descriptivos, coordenadas, fecha, comentarios, versión de taxonomía y estado de sincronización. Los adjuntos se modelan como recursos independientes para conservar metadatos y permitir una gestión posterior más flexible.

T09 depende de T05, porque toda observación debe pertenecer a una sesión, y de T07, porque el tipo de residuo procede de la taxonomía. Su impacto en el producto final es convertir el backend en receptor de los datos científicos de campo, preparando la posterior visualización en el portal, la sincronización y las exportaciones.

T10 – App móvil: formulario de observación

La décima tarea completa el primer flujo funcional de campo al incorporar en la aplicación móvil el formulario de observación. Su objetivo funcional es permitir que el usuario registre cada residuo observado con los datos mínimos necesarios para su análisis posterior.

La memoria recoge este flujo como una secuencia en la que el investigador selecciona un elemento de la taxonomía, introduce cantidad y atributos complementarios, añade comentarios y, cuando procede, incorpora una fotografía geolocalizada. El formulario se apoya en la navegación y en la sesión activa de la app móvil, y se relaciona con el backend a través de la API de observaciones y adjuntos. En un escenario *offline-first*, los registros se almacenan localmente y se preparan para su sincronización diferida.

T10 depende de T07, T08 y T09, ya que necesita taxonomía, sesión activa y una API de persistencia para consolidar los datos. Su impacto en el producto final es especialmente visible: representa el punto en el que OmniLitter pasa de ser una estructura de gestión a una herramienta efectiva de captura de datos científicos en campo, dejando preparada la base para módulos posteriores como mapas, análisis, exportación y sincronización avanzada.

5.3.0.2. Sprint 2: Captura avanzada de observaciones y flujo offline

Durante el segundo sprint se abordaron las funcionalidades necesarias para completar el flujo de trabajo de campo en la aplicación móvil y permitir la posterior explotación de los datos desde el portal web. Este sprint se centró especialmente en la captura precisa de observaciones, el soporte offline, la gestión de fotografías y la validación de la información registrada.

Cuadro 5.2: Resumen del backlog del Sprint 2: captura avanzada y flujo offline.

ID	Funcionalidad	Área	Dependencias
T11	App móvil: GPS en sesión y observación	App móvil	T08, T10
T12	App móvil: fotos por observación y metadatos	App móvil y backend	T09, T10, T11
T13	App móvil: revisión, edición y eliminación antes de sincronizar	App móvil	T08, T10, T12
T14	Backend: validación científica y permisos de observaciones	Backend	T09, T10, T11, T12
T15	App móvil: persistencia offline del flujo de campo	App móvil	T07, T08, T10, T11, T12
T16	Portal web: listado de sesiones y observaciones con filtros	Web y backend	T04, T05, T09, T14
T17	Portal web: ficha de observación con fotos	Web	T12, T16

T11 – GPS en sesión y observación

Área	App móvil
Responsable	Ignacio Martínez Díaz
Prioridad	Crítica
Dependencias	T08, T10

Objetivo

Integrar la captura de coordenadas GPS dentro del flujo de sesiones y observaciones, permitiendo registrar la localización exacta tanto del inicio de la sesión de muestreo como de cada observación individual.

Valor aportado

Permite georreferenciar la información científica recopilada y facilita su análisis posterior mediante herramientas cartográficas.

T12 – Fotos por observación y metadatos

Área	App móvil + Backend
Responsable	Ignacio Martínez Díaz
Prioridad	Alta
Dependencias	T09, T10, T11

Objetivo

Permitir la captura y asociación de fotografías a cada observación registrada, almacenando además metadatos relevantes como tamaño, formato, dimensiones, fecha de captura y otra información técnica asociada.

Valor aportado

Aporta evidencia visual a las observaciones y mejora los procesos de validación científica posteriores.

T13 – Revisión, edición y eliminación antes de sincronizar

Área	App móvil
Responsable	Ignacio Martínez Díaz
Prioridad	Alta
Dependencias	T08, T10, T12

Objetivo

Permitir al usuario revisar las observaciones almacenadas localmente antes de sincronizarlas con el servidor, incorporando capacidades de edición y eliminación.

Valor aportado

Mejora la calidad de los datos enviados al sistema central y reduce errores producidos durante la captura en campo.

T14 – Validación científica y permisos de observaciones

Área	Backend
Responsable	Alejandro
Prioridad	Crítica
Dependencias	T09, T10, T11, T12

Objetivo

Implementar mecanismos de validación y autorización que garanticen la consistencia de los datos científicos y el cumplimiento de los permisos definidos para cada rol.

Valor aportado

Protege la integridad de la información registrada y evita modificaciones no autorizadas dentro del sistema.

T15 – Persistencia offline del flujo de campo

Área	App móvil
Responsable	Ignacio Martínez Díaz
Prioridad	Crítica
Dependencias	T07, T08, T10, T11, T12

Objetivo

Garantizar el funcionamiento completo de la aplicación móvil en ausencia de conectividad mediante almacenamiento local de sesiones, observaciones, fotografías y coordenadas GPS.

Valor aportado

Permite utilizar OmniLitter en entornos remotos donde no existe cobertura de red estable, manteniendo la continuidad del trabajo de campo.

T16 – Listado de sesiones y observaciones con filtros

Área	Web + Backend
Responsable	Alejandro
Prioridad	Alta
Dependencias	T04, T05, T09, T14

Objetivo

Desarrollar una interfaz web que permita consultar sesiones y observaciones registradas mediante filtros y mecanismos de búsqueda avanzados.

Valor aportado

Facilita el acceso a la información recopilada y mejora las capacidades de análisis por parte de investigadores y administradores.

T17 – Ficha de observación con fotografías

Área	Web
Responsable	Alejandro
Prioridad	Alta
Dependencias	T12, T16

Objetivo

Implementar una vista detallada de observación en el portal web que permita consultar todos los atributos registrados junto con las fotografías asociadas.

Valor aportado

Facilita la validación visual de los registros y completa el ciclo de captura, sincronización y revisión de observaciones.

5.4. Gestión de reuniones

Las reuniones con el cliente han tenido un papel central en la definición del alcance. La primera reunión permitió identificar el problema y las funcionalidades esperadas. Posteriormente, una segunda reunión sirvió para revisar los mockups, confirmar la cobertura de los flujos principales y reforzar prioridades como el funcionamiento sin conexión, la separación por grupos y la trazabilidad científica.

Estas reuniones se documentan en la Sección 4 y en el Anexo B, donde se relacionan las necesidades detectadas con los requisitos del sistema.

5.5. Gestión de riesgos

5.6. Gestión del alcance

El alcance se ha dividido en dos niveles:

- **Alcance funcional validado mediante mockups:** portal web y aplicación móvil con flujos navegables, datos de ejemplo, roles simulados y pantallas principales implementadas.
- **Alcance de implementación final:** backend REST, base de datos, autenticación, sincronización offline, persistencia de observaciones y conexión progresiva de los clientes al backend.

Esta separación permite demostrar primero la experiencia de uso y después construir la infraestructura técnica necesaria para convertir el prototipo en una aplicación real.

6. Tecnologías utilizadas

En este capítulo se describen las tecnologías seleccionadas para el desarrollo de OmniLitter y la justificación de su elección. La selección se ha realizado teniendo en cuenta tres restricciones principales: el alcance de un Trabajo de Fin de Grado desarrollado por dos personas, la necesidad de disponer de una aplicación móvil capaz de trabajar sin conexión y la conveniencia de mantener un stack homogéneo basado en TypeScript.

6.1. Criterios de selección

Los criterios utilizados para seleccionar las tecnologías han sido los siguientes:

- **Homogeneidad tecnológica:** se prioriza el uso de TypeScript tanto en el frontend como en el backend para reducir el cambio de contexto durante el desarrollo.
- **Capacidad offline:** la aplicación móvil debe poder crear sesiones, observaciones y fotografías sin depender de la conexión a internet.
- **Madurez del ecosistema:** se eligen tecnologías ampliamente usadas, con documentación suficiente y comunidad activa.
- **Facilidad de despliegue:** el sistema debe poder desplegarse en servicios accesibles para un proyecto académico, como Vercel, Render, Railway o una máquina virtual sencilla.
- **Escalabilidad razonable:** aunque el proyecto no persigue una escala industrial, el diseño debe permitir incorporar nuevos grupos, campañas y datos de campo sin rehacer la arquitectura.

Tecnologías empleadas en los mockups Durante la fase inicial se han construido dos prototipos funcionales con datos en memoria: un portal web y una simulación de aplicación móvil en navegador. Ambos prototipos permiten validar flujos de usuario con el cliente antes de desarrollar la versión final.

Cuadro 6.1: Tecnologías empleadas en los mockups funcionales.

Tecnología	Uso en el proyecto
React 18	Construcción de interfaces de usuario para el portal web y el mockup móvil.
TypeScript	Tipado estático de componentes, rutas, estado y datos de ejemplo.
Vite	Servidor de desarrollo y empaquetado rápido de los prototipos.
Tailwind CSS	Sistema de estilos utilitario para construir pantallas responsive.
React Router	Definición de rutas públicas, rutas protegidas y navegación entre pantallas.
Recharts	Gráficas y visualizaciones estadísticas del dashboard del portal web.
Lucide React / Material Icons	Iconografía de la interfaz.

6.2. Portal web

El portal web definitivo se plantea como una aplicación *Single Page Application* desarrollada con React, TypeScript y Vite. Su responsabilidad principal será permitir a investigadores y administradores consultar observaciones, gestionar usuarios y grupos, visualizar mapas y exportar datos.

Cuadro 6.2: Stack previsto para el portal web.

Tecnología	Responsabilidad
React	Construcción de componentes reutilizables y pantallas del portal.
TypeScript	Definición de tipos de dominio, respuestas de API y props de componentes.
Vite	Compilación, servidor de desarrollo y generación del bundle final.
Tailwind CSS	Sistema visual responsive y consistente con los mockups.
React Router	Gestión de rutas públicas y protegidas.
Axios / Fetch API	Comunicación con la API REST del backend.
Leaflet / React Leaflet	Visualización geográfica de observaciones en mapas interactivos.
Recharts	Dashboards, rankings y tendencias temporales.

6.3. Aplicación móvil

La aplicación móvil se plantea con React Native y Expo, utilizando SQLite mediante el paquete `expo-sqlite` como base de datos local. El objetivo es que el usuario pueda completar una jornada de muestreo aunque el dispositivo no tenga conexión a internet.

Cuadro 6.3: Stack previsto para la aplicación móvil.

Tecnología	Responsabilidad
React Native	Desarrollo multiplataforma para Android e iOS.
Expo	Toolchain de desarrollo, compilación y acceso a módulos nativos.
TypeScript	Tipado de pantallas, servicios, modelos locales y respuestas de API.
<code>expo-sqlite</code> / SQLite	Persistencia local de sesiones, observaciones, fotografías pendientes y cola de sincronización.
<code>expo-location</code>	Captura de coordenadas GPS asociadas a sesiones y observaciones.
<code>expo-camera</code>	Captura de fotografías y obtención de metadatos cuando sea posible.
<code>expo-task-manager</code> / <code>expo-background-task</code>	Ejecución de tareas de sincronización en segundo plano cuando el sistema operativo lo permita.
React Navigation	Navegación entre pantallas principales y flujos de captura.

6.4. Backend y persistencia

El backend centralizará la lógica de negocio, la autenticación, el control de acceso por grupo y la sincronización de datos procedentes de dispositivos móviles. Se plantea como una API REST desarrollada con Node.js, Express y TypeScript.

Cuadro 6.4: Stack previsto para backend y persistencia.

Tecnología	Responsabilidad
Node.js	Entorno de ejecución del backend.
Express	Definición de rutas, middlewares y controladores REST.
TypeScript	Tipado de servicios, DTOs, entidades y errores.
PostgreSQL	Base de datos relacional principal del sistema.
Prisma	ORM para modelar entidades, migraciones y acceso a datos.
JWT	Autenticación mediante tokens de acceso y refresco.
bcrypt	Almacenamiento seguro de contraseñas mediante hash.
multer / almacenamiento de ficheros	Recepción y almacenamiento de fotografías asociadas a observaciones.

6.5. Despliegue y herramientas auxiliares

6.6. Alternativas consideradas

Justificación de SQLite frente a WatermelonDB Aunque inicialmente se valoró WatermelonDB como solución local *offline-first*, se ha decidido emplear SQLite directamente para ajustar mejor el alcance del TFG. SQLite permite cubrir los requisitos principales de la aplicación móvil: almacenar sesiones, observaciones, fotografías pendientes y una cola de sincronización local.

La decisión reduce la complejidad técnica, evita depender de una capa adicional de sincronización y facilita que los autores controlen de forma explícita el esquema local y el flujo de subida de datos. El patrón *offline-first* se mantiene, pero se implementa mediante una tabla local de cola de sincronización y servicios propios de lectura/escritura sobre SQLite.

7. Diseño de la solución

Este capítulo describe las decisiones de diseño adoptadas para materializar los requisitos funcionales y no funcionales de OmniLitter en una arquitectura concreta. Se abordan, en orden de abstracción descendente, la arquitectura general del sistema, el diseño de la base de datos, el contrato de la API REST, la estructura de componentes del portal web y de la aplicación móvil, y el mecanismo de sincronización *offline*.

7.1. Arquitectura general

OmniLitter sigue una arquitectura de **tres capas** clásica adaptada a un contexto multi-plataforma con requisito de operación sin conexión:

1. **Capa de presentación.** Comprende dos clientes independientes: el portal web, orientado a la consulta y gestión de datos por parte de investigadores y administradores, y la aplicación móvil, orientada a la captura de observaciones en campo.
2. **Capa de lógica de negocio.** Una API REST centralizada planteada con Node.js + Express actúa como único punto de acceso a los datos, aplica las reglas de negocio y gestiona la autenticación mediante *tokens* JWT.
3. **Capa de persistencia.** PostgreSQL almacena el estado canónico del sistema. Adicionalmente, la aplicación móvil mantiene una base de datos local SQLite que permite operar sin conexión y sincronizarse en diferido.

La figura 7.1 muestra un diagrama de la arquitectura del sistema.

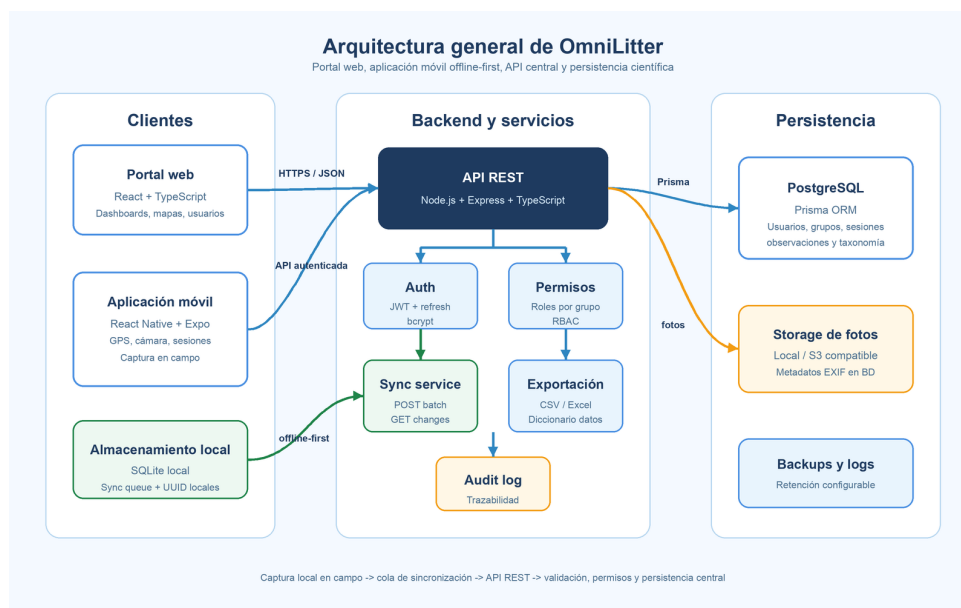


Figura 7.1: Arquitectura general propuesta para OmniLitter.

7.1.0.1. Justificación de las decisiones arquitectónicas principales

API REST como único punto de acceso. Centralizar la lógica de negocio en la API evita duplicidades entre los dos clientes y facilita la incorporación de nuevos consumidores en el futuro (p.ej., exportaciones automatizadas por *scripts* externos o integraciones con repositorios científicos).

Separación entre portal web y aplicación móvil. Aunque comparten la misma API, los dos clientes tienen requisitos de experiencia de usuario muy distintos: el portal prioriza la visualización analítica y la gestión, mientras que la app móvil prioriza la captura rápida en condiciones de campo adversas (luz solar, guantes, sin conexión). Mantenerlos como proyectos independientes permite evolucionar cada uno a su propio ritmo.

Base de datos local en el móvil. El trabajo de campo se realiza habitualmente en zonas costeras, ribereñas o forestales con cobertura de red intermitente o nula. La decisión de embeber una base de datos local en el dispositivo es, por tanto, un requisito no negociable para la viabilidad del producto.

7.2. Diseño de la base de datos

7.2.0.1. Modelo relacional

El modelo de datos de OmniLitter se compone de diez entidades que se agrupan en tres áreas funcionales:

- **Gestión de acceso:** USER, GROUP, MEMBERSHIP.

- **Organización científica:** CAMPAIGN, SURVEY_SESSION.
- **Datos de campo:** OBSERVATION, PHOTO, TAXONOMY_ITEM, FAVORITE, AUDIT_LOG.

La figura 7.2 muestra el diagrama entidad-relación del sistema.

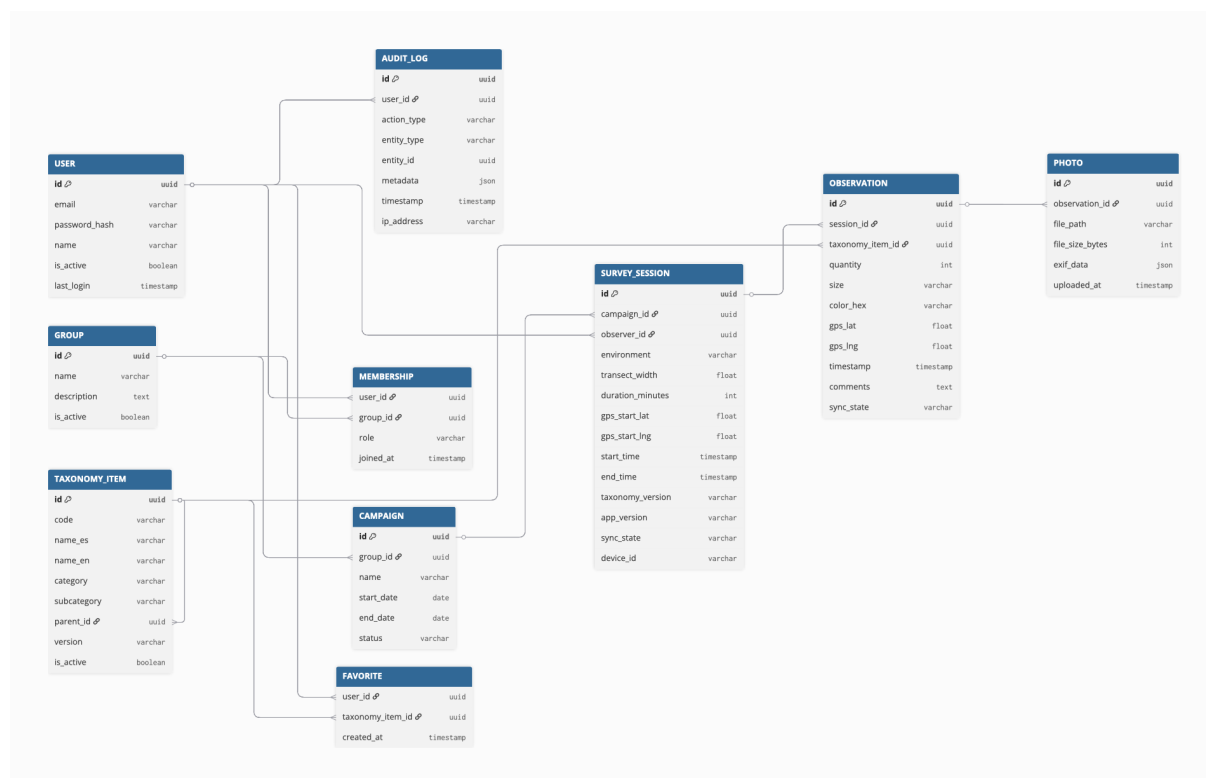


Figura 7.2: Diagrama entidad-relación del sistema OmniLitter.

7.2.0.2. Descripción de entidades

La tabla 7.1 describe los atributos clave de cada entidad y su propósito en el sistema.

Cuadro 7.1: Descripción de entidades del modelo de datos.

Entidad	Atributos principales	Descripción
USER	id (UUID), email, password_hash, name, is_active, last_login	Representa a cada persona registrada en el sistema. Las contraseñas se almacenan como <i>hash</i> bcrypt y nunca en texto plano.
GROUP	id (UUID), name, description, is_active	Equipo de investigación. Un grupo puede ser archivado sin eliminar sus datos históricos, preservando la trazabilidad científica.

Entidad	Atributos principales	Descripción
MEMBERSHIP	user_id (FK), group_id (FK), role (admin editor viewer), joined_at	Tabla de unión que asocia usuarios con grupos asignándoles un rol. Un mismo usuario puede tener roles distintos en grupos distintos.
CAMPAIGN	id (UUID), group_id (FK), name, start_date, end_date, status	Proyecto científico de muestreo. Agrupa varias sesiones de campo bajo un objetivo común (p.ej., «Monitoreo anual Playa de la Caleta 2026»).
SURVEY_SESSION	id (UUID), campaign_id (FK), observer_id (FK), environment, transect_width, duration_minutes, gps_start_lat/lng, start_time, end_time, taxonomy_version, app_version, sync_state, device_id	Sesión de muestreo individual en campo. Registra todos los metadatos científicos necesarios para la reproducibilidad: protocolo, versión de taxonomía utilizada y versión de la aplicación. El campo <code>sync_state</code> gestiona el ciclo de vida de la sincronización.
OBSERVATION	id (UUID), session_id (FK), taxonomy_item_id (FK), quantity, size, color_hex, gps_lat/lng, timestamp, taxonomy_version, comments, sync_state	Registro de un ítem de residuo observado. Constituye la unidad mínima de dato científico del sistema.
PHOTO	id (UUID), observation_id (FK), file_path, file_size_bytes, exif_data (JSONB), uploaded_at	Imagen asociada a una observación. Los metadatos EXIF (coordenadas GPS embebidas en la imagen, fecha de captura, modelo de cámara) se almacenan en formato JSONB para máxima flexibilidad.

Entidad	Atributos principales	Descripción
TAXONOMY_ITEM	id (UUID), code, name_es, name_en, category, subcategory, parent_id (FK self-ref), version, is_active	Catálogo de tipos de residuo basado en la <i>Joint List of Litter Categories</i> . La estructura jerárquica auto-referenciada (categoría → subcategoría → ítem) permite navegar la taxonomía sin conexión, ya que se descarga completa al iniciar sesión.
FAVORITE	user_id (FK), taxonomy_item_id (FK), created_at	Ítems de taxonomía marcados como favoritos por el usuario para agilizar la captura de observaciones en campo.
AUDIT_LOG	id, user_id (FK), action_type, entity_type, entity_id, metadata (JSONB), timestamp, ip_address	Registro de auditoría que almacena las acciones relevantes sobre el sistema (inicio de sesión, creación y edición de observaciones, exportaciones de datos, cambios de configuración). Garantiza la trazabilidad exigida en entornos de investigación científica.

7.2.0.3. Decisiones de diseño de la base de datos

UUID como clave primaria en registros sincronizables. Los registros que pueden crearse en el dispositivo móvil antes de comunicarse con el servidor utilizan UUID v4 generados en el cliente. Esto resuelve el problema de colisiones de claves en escenarios de creación offline: dos dispositivos diferentes que creen una observación sin conexión obtendrán identificadores globalmente únicos que no entrarán en conflicto cuando se sincronicen con el servidor.

Rol en MEMBERSHIP, no en USER. El rol de un investigador no es una propiedad intrínseca de la persona, sino de su relación con un grupo concreto. Un mismo investigador puede ser administrador en su grupo principal y colaborador invitado en otro grupo. Modelar el rol en la tabla de unión es, por tanto, la decisión correcta desde el punto de vista semántico.

Versión de taxonomía en sesiones y observaciones. La taxonomía de residuos evoluciona con el tiempo a medida que la comunidad científica añade o reorganiza categorías. Registrar la versión exacta utilizada en cada sesión permite reconstruir con exactitud qué definición se aplicó a cada observación, lo que es un requisito básico de reproducibilidad científica.

JSONB para metadatos heterogéneos. Los campos `exif_data` y `metadata` utilizan el

tipo JSONB de PostgreSQL para almacenar datos cuya estructura puede variar (distintos modelos de cámara incluyen campos EXIF diferentes, distintas acciones de auditoría tienen distintos contextos). Esta decisión evita la proliferación de columnas opcionales con valor nulo y preserva la flexibilidad para añadir nuevos atributos sin migrar el esquema.

7.3. Diseño de la API REST

La API sigue los principios REST: interfaz uniforme, comunicación sin estado, recursos identificados por URI y representaciones en formato JSON. Todos los *endpoints* protegidos requieren el encabezado **Authorization: Bearer <token>**, donde el *token* es un JWT firmado emitido en el *endpoint* de autenticación.

La tabla 7.2 recoge los *endpoints* principales agrupados por recurso.

Cuadro 7.2: Principales *endpoints* de la API REST de OmniLitter.

Método	Ruta	Protegido	Descripción
<i>Autenticación</i>			
POST	/api/auth/login	No	Emite JWT de acceso y refresco
POST	/api/auth/refresh	No	Renueva el JWT de acceso
POST	/api/auth/logout	Sí	Invalida el <i>token</i> de refresco
<i>Usuarios</i>			
GET	/api/users	Sí (admin)	Lista todos los usuarios
POST	/api/users	Sí (admin)	Crea un nuevo usuario
GET	/api/users/:id	Sí	Obtiene un usuario
PATCH	/api/users/:id	Sí (admin)	Actualiza datos o estado
<i>Grupos</i>			
GET	/api/groups	Sí	Lista grupos del usuario autenticado
POST	/api/groups	Sí (admin)	Crea un grupo
PATCH	/api/groups/:id	Sí (admin)	Edita nombre/descripción
DELETE	/api/groups/:id/archive	Sí (admin)	Archiva un grupo
POST	/api/groups/:id/members	Sí (admin)	Añade miembro
DELETE	/api/groups/:id/members/:uid	Sí (admin)	Elimina miembro

Método	Ruta	Protegido	Descripción
<i>Sesiones de muestreo</i>			
GET	/api/sessions	Sí	Lista sesiones accesibles
POST	/api/sessions	Sí	Crea una sesión
GET	/api/sessions/:id	Sí	Obtiene una sesión con observaciones
PATCH	/api/sessions/:id	Sí	Actualiza metadatos
<i>Observaciones</i>			
GET	/api/sessions/:id/observations	Sí	Lista observaciones de sesión
POST	/api/sessions/:id/observations	Sí	Registra una observación
GET	/api/observations	Sí	Lista todas (filtros opcionales)
<i>Fotos</i>			
POST	/api/observations/:id/photos	Sí	Sube una foto (multipart)
GET	/api/photos/:id	Sí	Sirve el archivo de imagen
<i>Taxonomía</i>			
GET	/api/taxonomy	Sí	Descarga la taxonomía completa
GET	/api/taxonomy?version=:v	Sí	Descarga versión específica
<i>Sincronización</i>			
POST	/api/sync/batch	Sí	Sube cola de registros pendientes
GET	/api/sync/changes	Sí	Descarga cambios desde last_sync
<i>Exportación</i>			
GET	/api/export/csv	Sí (admin)	Exporta observaciones en CSV científico

7.4. Seguridad y control de acceso

El sistema se diseña con autenticación basada en JWT y dos *tokens*: un **token de acceso** de corta duración (15 minutos) y un **token de refresco** de larga duración (7 días). El

token de acceso se envía en cada petición en el encabezado **Authorization**; el de refresco se utiliza exclusivamente para solicitar un nuevo token de acceso cuando el anterior expira.

La autorización sigue un modelo de control de acceso basado en roles (RBAC) a nivel de grupo. El *middleware* de autorización verifica, para cada petición, que el usuario autenticado posee el rol requerido en el grupo al que pertenece el recurso solicitado. Esto permite que una misma persona sea administradora en un grupo y colaboradora en otro, con permisos diferentes en cada contexto.

7.5. Diseño del portal web

7.5.0.1. Arquitectura de componentes

El portal web se diseña como una *Single Page Application* (SPA) con React 18 y TypeScript, empaquetada con Vite y estilizada con Tailwind CSS. La navegación entre páginas es gestionada por React Router, que define dos niveles de rutas:

- **Rutas públicas:** la página de inicio (/) y el formulario de inicio de sesión (/login), accesibles sin autenticación.
- **Rutas protegidas:** todas las páginas bajo /app/*, protegidas por un componente `PrivateRoute` que verifica el estado de autenticación y redirige al *login* si no existe sesión activa.

La estructura de componentes sigue una jerarquía de tres niveles: el **Layout** general (barra lateral, encabezado, selector de grupo activo) envuelve las páginas individuales, que a su vez utilizan componentes reutilizables (tarjetas de estadística, tablas filtrables, modales de confirmación).

7.5.0.2. Gestión de estado

En el prototipo del portal web, `localStorage` se utiliza como mecanismo de persistencia de la sesión del usuario entre recargas de página, almacenando el estado necesario para simular autenticación, rol y grupo activo. En la implementación final, este comportamiento deberá integrarse con los *tokens* emitidos por el backend y con una estrategia de refresco de sesión.

En el mockup actual, la comunicación entre el selector de grupo activo (ubicado en la barra lateral) y el componente Dashboard se resuelve mediante un evento personalizado del DOM (`omnilitter:groupChanged`). Esta solución es suficiente para el prototipo y podrá sustituirse por un contexto global o una librería de estado si la implementación final aumenta de complejidad.

7.5.0.3. Flujo de navegación y control de acceso por rol

Las secciones de **Grupos** y **Usuarios** son exclusivas del rol de administrador y no aparecen en la barra de navegación para los colaboradores. El Dashboard ajusta el contenido mostrado según el rol: los administradores ven estadísticas globales del grupo activo, mientras que los colaboradores ven únicamente sus propias métricas. La tabla de observaciones filtra los registros por autor para los colaboradores y muestra todos para los administradores.

7.6. Diseño de la aplicación móvil

7.6.0.1. Stack tecnológico y estructura de pantallas

La aplicación móvil está planteada con **React Native** y **Expo**, usando *development builds* o *prebuild* cuando sea necesario acceder a capacidades nativas del dispositivo. Esta aproximación permite mantener las ventajas del ecosistema Expo sin renunciar a módulos como `expo-location`, `expo-camera` y las tareas de sincronización en segundo plano mediante `expo-task-manager` y `expo-background-task`.

La navegación está organizada en dos niveles:

- **Navegación de primer nivel (*tab bar*):** cinco pestañas inferiores (Inicio, Sesiones, Mapa, Sincronización, Perfil), visibles únicamente en las pantallas principales.
- **Navegación de pila (*stack*):** flujos de varios pasos como la creación de una nueva sesión o la captura de una observación, que se presentan sin el *tab bar*.

7.6.0.2. Gestión de estado global

El estado global de la aplicación se gestiona mediante el patrón **Context + useReducer** de React, sin dependencias externas de gestión de estado. El contexto `AppContext` expone el estado completo (sesiones activas, tipos de residuo, datos del usuario, estado de sincronización) y un *dispatcher* con acciones tipadas en TypeScript. Este patrón proporciona la predictibilidad de Redux con una superficie de código significativamente menor, adecuada para la escala del proyecto.

7.6.0.3. Base de datos local con SQLite

SQLite, integrado en Expo mediante `expo-sqlite`, actúa como base de datos local de la aplicación móvil. Se elige por ser una solución madura, ligera y suficientemente potente para almacenar el volumen de datos esperado durante una jornada de muestreo. Además,

permite mantener control directo sobre el esquema local y sobre la cola de sincronización, reduciendo la complejidad respecto a incorporar una capa adicional de abstracción.

El esquema local replica el subconjunto del modelo de datos necesario para la operación en campo: `survey_sessions`, `observations`, `photos` y `sync_queue`, además de la taxonomía completa cargada al iniciar sesión. La tabla `sync_queue` registra las operaciones pendientes de subida al servidor junto con su *payload*, número de intentos y estado de sincronización.

7.6.0.4. Flujo de captura de una observación

El flujo principal de la aplicación sigue esta secuencia:

1. El investigador inicia una nueva sesión de muestreo, especificando el nombre, el tipo de entorno y los parámetros del transecto.
2. Para cada ítem encontrado, abre la pantalla de nueva observación, selecciona el tipo de residuo a través del modal de taxonomía (organizado en cuadrícula visual por categorías e iconos), e introduce cantidad, tamaño, color y comentarios.
3. Opcionalmente captura una foto geolocalizada del residuo.
4. Al finalizar la sesión de campo, revisa el resumen y puede iniciar la sincronización manualmente o dejar que se realice en segundo plano al recuperar cobertura de red.

7.7. Sincronización offline

7.7.0.1. Patrón Outbox Queue

El mecanismo de sincronización se diseña siguiendo el patrón **Outbox Queue** (cola de salida), ampliamente utilizado en sistemas distribuidos con conectividad intermitente. El funcionamiento es el siguiente:

1. Cuando el usuario crea o modifica un registro (sesión u observación) en el dispositivo sin conexión, el registro se persiste en SQLite con `sync_state = PENDING` y se inserta una entrada en la tabla `sync_queue` con el *payload* completo del cambio.
2. Cuando el dispositivo recupera conectividad, una tarea de sincronización en segundo plano o una acción manual del usuario lee la cola y realiza una petición `POST /api/sync/batch` con todos los registros pendientes.
3. El servidor procesa los registros en orden, los persiste en PostgreSQL y devuelve el estado canónico actualizado.
4. El cliente actualiza el `sync_state` de los registros procesados a `SYNCED` y descarga los cambios remotos realizados por otros usuarios mediante el endpoint `GET`

/api/sync/changes, enviando `last_sync_timestamp` como parámetro de consulta.

5. En caso de error en un registro concreto (conflicto de datos, registro inválido), ese registro pasa a `sync_state = ERROR` y se notifica al usuario, sin bloquear la sincronización del resto.

7.7.0.2. Estados de sincronización

Cada sesión y observación puede encontrarse en uno de cuatro estados, como muestra la tabla 7.3.

Cuadro 7.3: Estados de sincronización y su significado.

Estado	Significado
PENDING	El registro existe solo en el dispositivo; aún no se ha intentado sincronizar.
SYNCING	El registro está siendo procesado en la petición de sincronización actual.
SYNCED	El servidor ha confirmado la recepción y persistencia del registro.
ERROR	El servidor rechazó el registro (conflicto, datos inválidos). Requiere intervención manual o reintento.

7.7.0.3. Resolución de conflictos

La estrategia de resolución de conflictos propuesta es **last-write-wins** con granularidad de registro: si dos usuarios modifican la misma sesión de forma independiente mientras están sin conexión, prevalece la versión con el `timestamp` más reciente. Esta estrategia es suficiente para el modelo de uso de OmniLitter, donde cada sesión de muestreo pertenece a un único observador responsable y los conflictos genuinos son infrecuentes.

7.7.0.4. Garantía de identificadores únicos en entorno offline

Para que la estrategia Outbox Queue funcione sin depender de un identificador asignado por el servidor en el momento de la captura, los registros creados en modo offline usan **UUID v4 generados en el cliente**. Esto garantiza que las sesiones y observaciones creadas simultáneamente en distintos dispositivos no colisionarán al sincronizarse, independientemente del orden de llegada al servidor.

8. Implementación

8.1. Estructura del repositorio

8.2. Implementación del backend

8.3. Implementación del portal web

8.4. Implementación de la aplicación móvil

8.5. Persistencia local y sincronización

8.6. Configuración de entornos

9. Pruebas y validación

9.1. Estrategia de pruebas

9.2. Pruebas funcionales

9.3. Pruebas de sincronización y offline

9.4. Pruebas de permisos y seguridad

9.5. Validación con cliente

9.6. Matriz requisito-prueba

10. Despliegue y operación

10.1. Arquitectura de despliegue

10.2. Configuración de producción

10.3. Despliegue de API y base de datos

10.4. Despliegue del portal web

10.5. Build y distribución de la aplicación móvil

10.6. Limitaciones operativas

11. Conclusiones y trabajo futuro

11.1. Cumplimiento de objetivos

11.2. Limitaciones

11.3. Lecciones aprendidas

11.4. Líneas de trabajo futuro

Bibliografía

- [1] Expo. Expo documentation. <https://docs.expo.dev>. Último acceso: mayo 2026.
- [2] Meta Open Source. React documentation. <https://react.dev>. Último acceso: mayo 2026.
- [3] Meta Open Source. React native documentation. <https://reactnative.dev>. Último acceso: mayo 2026.
- [4] OpenJS Foundation. Express documentation. <https://expressjs.com>. Último acceso: mayo 2026.
- [5] Prisma Data, Inc. Prisma documentation. <https://www.prisma.io/docs>. Último acceso: mayo 2026.
- [6] SQLite Consortium. Sqlite documentation. <https://www.sqlite.org/docs.html>. Último acceso: mayo 2026.
- [7] The PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>. Último acceso: mayo 2026.
- [8] Vite. Vite documentation. <https://vite.dev>. Último acceso: mayo 2026.

A. Declaración del uso de IA

En cumplimiento con las buenas prácticas académicas y de transparencia, se declara que para la elaboración de este trabajo se ha contado con el apoyo de herramientas de Inteligencia Artificial Generativa. El uso de dichas herramientas se ha limitado a tareas de apoyo, contraste y revisión, manteniendo los autores la responsabilidad final sobre las decisiones técnicas, el código entregado y la redacción definitiva de la memoria.

A.1. Usos realizados

Las herramientas de IA se han utilizado principalmente en las siguientes tareas:

- **Organización inicial del trabajo.** Apoyo para estructurar la memoria, ordenar capítulos y decidir qué artefactos documentales eran necesarios.
- **Contraste de stack tecnológico.** Comparación de alternativas para el portal web, la aplicación móvil, el backend, la base de datos y el mecanismo de sincronización offline.
- **Prototipado de interfaces.** Ayuda en la generación y ajuste de mockups funcionales del portal web y de la aplicación móvil.
- **Diseño de arquitectura.** Apoyo en la elaboración de diagramas, separación de componentes, flujo de sincronización y modelo de datos.
- **Revisión de texto.** Reformulación y mejora de redacción de apartados de la memoria, siempre sobre contenido revisado por los autores.

El Cuadro A.1 resume las partes del trabajo donde se utilizó IA y con qué propósito.

Cuadro A.1: Detalle del uso de IA en el trabajo.

Parte del trabajo	Herramienta	Propósito
Estructura de memoria	Claude / Codex	Proponer organización de capítulos y anexos.
Stack tecnológico	Claude / Codex	Comparar alternativas y razonar decisiones técnicas.
Mockups funcionales	Claude / Codex	Apoyar la generación y corrección de pantallas React.
Diseño del sistema	Claude / Codex	Redactar borradores, diagramas y tablas técnicas.
Revisión de redacción	Claude / Codex	Mejorar claridad, coherencia y estilo académico.

A.2. Ejemplos de prompts utilizados

A continuación se recogen ejemplos representativos de prompts empleados durante el proyecto. No se incluyen todas las conversaciones, sino aquellos tipos de consulta que reflejan el uso realizado.

Elección del stack

Estamos diseñando una aplicación móvil de campo que debe funcionar sin conexión, capturar GPS y fotos, y sincronizar observaciones con un backend cuando recupere conectividad. Compara alternativas para la base de datos local y razona si tiene sentido usar SQLite directamente en este contexto.

Diseño de arquitectura

Necesito plantear el capítulo de diseño de la solución para una plataforma formada por portal web, app móvil offline-first y backend REST. Propone una estructura para explicar arquitectura, componentes, base de datos y sincronización.

Revisión de requisitos

Tenemos requisitos obtenidos en reuniones con el cliente y los hemos contrastado mediante mockups. Ayúdame a organizarlos en requisitos funcionales, no funcionales y de datos científicos, manteniendo trazabilidad con las reuniones.

Revisión de redacción

He redactado un apartado de la memoria y quiero que suene más claro y técnicamente correcto. Reformula el texto manteniendo el contenido y las decisiones tomadas por el equipo.

A.3. Autoría y revisión crítica

Los autores declaran conocer y poder explicar el contenido incorporado al trabajo. Las respuestas obtenidas de las herramientas de IA se trataron como material de apoyo y fueron revisadas críticamente antes de su inclusión. La selección del dominio, la validación con el cliente, la toma de decisiones, la implementación final, la ejecución de pruebas y la entrega definitiva son responsabilidad de los autores.

B. Requisitos detallados y validación con cliente

Este anexo recoge de forma más detallada la información utilizada para construir el registro de requisitos del proyecto y el proceso de validación seguido con el cliente. Su objetivo es dejar constancia del origen de las decisiones de diseño y de la evolución del alcance tras contrastar la propuesta inicial.

B.1. Resumen de reuniones con el cliente

Cuadro B.1: Resumen de reuniones mantenidas con el cliente.

Reunión	Objetivo	Resultado principal
Toma inicial de requisitos	Comprender el problema, el alcance esperado y las limitaciones de la herramienta existente.	Se identificó la necesidad de una solución formada por aplicación móvil, portal web, funcionamiento offline, grupos de usuarios, mapas, dashboards y taxonomía de residuos.
Revisión de requisitos y mockups	Contrastar los requisitos definidos con los prototipos interactivos del portal y la aplicación móvil.	Se validó que los mockups cubrían los procesos principales y se reforzó la prioridad de la captura offline, la separación por grupos y la trazabilidad de los datos científicos.

B.2. Requisitos obtenidos en la reunión inicial

Durante la primera reunión se extrajeron los siguientes requisitos de alto nivel:

1. Desarrollar una plataforma compuesta por aplicación móvil y portal web.
2. Permitir registrar residuos en distintos entornos naturales, no solo ríos y mares, sino también playas, bosques y otros terrenos.

3. Registrar información científica mínima de cada observación: tipo de residuo, cantidad, posición, fecha, hora, entorno y comentarios.
4. Permitir adjuntar fotografías a las observaciones.
5. Garantizar que la aplicación móvil pueda utilizarse sin conexión a internet.
6. Sincronizar automáticamente los datos cuando el dispositivo recupere conectividad.
7. Gestionar usuarios, grupos y permisos para mantener la privacidad de los datos de cada equipo.
8. Ofrecer en el portal web herramientas de consulta, mapas, dashboards y exportación de datos.
9. Mantener una taxonomía de residuos jerárquica, navegable y ampliable.
10. Preservar trazabilidad científica mediante versiones de taxonomía, metadatos de sesión y registros de auditoría.

B.3. Validación de requisitos mediante mockups

La segunda reunión se centró en revisar los prototipos funcionales y comprobar que los flujos implementados representaban correctamente los requisitos ya recogidos. En lugar de generar un nuevo bloque de requisitos, esta revisión permitió confirmar la cobertura de los casos de uso principales:

- Inicio de sesión y diferenciación entre perfiles de administrador y colaborador.
- Selección de grupo activo y filtrado de la información asociada.
- Creación de sesiones de muestreo en la aplicación móvil.
- Registro de observaciones con taxonomía, cantidad, tamaño, color, comentarios, GPS y fotografía.
- Visualización de observaciones y sesiones desde el portal web.
- Consulta geográfica mediante mapas.
- Representación del estado de sincronización y de los datos pendientes.

Los ajustes visuales e internos realizados durante la construcción de los mockups se consideran decisiones de diseño del equipo y no requisitos adicionales solicitados por el cliente. Por ello no se documentan como cambios de alcance, sino como parte del proceso de refinamiento del prototipo.

B.4. Requisitos funcionales detallados

El registro de requisitos funcionales queda agrupado en seis bloques:

- **Autenticación y sesión:** inicio de sesión, cierre de sesión y recuperación de contraseña.
- **Roles, grupos y permisos:** gestión de grupos, usuarios, membresías y selección de grupo activo.
- **Captura de campo:** sesiones de muestreo, observaciones, GPS, fotografías, cantidad, tamaño, color y comentarios.
- **Taxonomía:** navegación jerárquica, búsqueda y favoritos.
- **Sincronización:** funcionamiento offline, cola de sincronización, sincronización manual y automática, y estados por elemento.
- **Portal web:** listados, filtros, mapas, heatmaps, dashboards, fotografías, exportación, grupos, usuarios y configuración.

B.5. Requisitos no funcionales detallados

Los requisitos no funcionales se agrupan en:

- **Seguridad:** comunicación cifrada, control de acceso por grupo y protección de adjuntos.
- **Fiabilidad:** sincronización reintentable y preservación de datos ante fallos o cierres inesperados.
- **Usabilidad:** interfaz adaptada a uso de campo en móvil y tablet.
- **Operación:** logs, copias de seguridad y matriz requisito-prueba.
- **Compatibilidad:** soporte para dispositivos móviles modernos, con especial atención a Android e iOS.

B.6. Requisitos de datos científicos

Los requisitos de datos científicos se centran en garantizar que las observaciones registradas puedan analizarse posteriormente sin perder contexto:

- Taxonomía jerárquica con códigos estables.
- Metadatos de muestreo por sesión.
- Metadatos de foto cuando estén disponibles.

- Extensión versionada de la taxonomía.
- Validaciones de obligatoriedad, rango y consistencia.
- Exportaciones con diccionario de columnas y unidades.
- Versión de taxonomía y de aplicación asociada a sesiones/exportaciones.

C. Manual de instalación y despliegue

D. Manual de usuario

E. Pruebas completas

F. API y diccionario de datos

G. Backlog completo